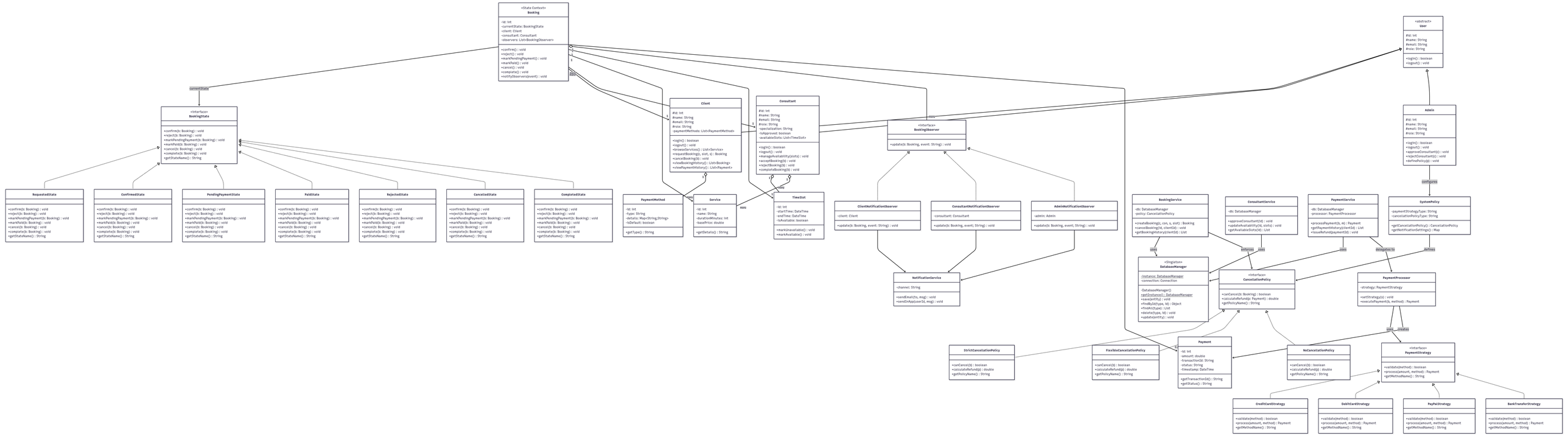
«State Context»
Booking
-Id: Int
-currentState: BookingState
-client: Client
-consultant: Consultant
-observers: List<BookingObserver>
+confirm() : void
+reject() : void
+markPendingPayment() : void
+markPaid() : void
+cancel() : void
+complete() : void
+notifyObservers(event) : void
currentState
«Interface»
BookingState
+confirm(b: Booking) : void
+reject(b: Booking) : void
+markPendingPayment(b: Booking) : void
+markPaid(b: Booking) : void
+cancel(b: Booking) : void
+complete(b: Booking) : void
+getStateName() : String
RequestedState
+confirm(b: Booking) : void
+reject(b: Booking) : void
+markPendingPayment(b: Booking) : void
+markPaid(b: Booking) : void
+cancel(b: Booking) : void
+complete(b: Booking) : void
+getStateName() : String
ConfirmedState
+confirm(b: Booking) : void
+reject(b: Booking) : void
+markPendingPayment(b: Booking) : void
+markPaid(b: Booking) : void
+cancel(b: Booking) : void
+complete(b: Booking) : void
+getStateName() : String
PendingPaymentState
+confirm(b: Booking) : void
+reject(b: Booking) : void
+markPendingPayment(b: Booking) : void
+markPaid(b: Booking) : void
+cancel(b: Booking) : void
+complete(b: Booking) : void
+getStateName() : String
PaidState
+confirm(b: Booking) : void
+reject(b: Booking) : void
+markPendingPayment(b: Booking) : void
+markPaid(b: Booking) : void
+cancel(b: Booking) : void
+complete(b: Booking) : void
+getStateName() : String
RejectedState
+confirm(b: Booking) : void
+reject(b: Booking) : void
+markPendingPayment(b: Booking) : void
+markPaid(b: Booking) : void
+cancel(b: Booking) : void
+complete(b: Booking) : void
+getStateName() : String
CanceledState
+confirm(b: Booking) : void
+reject(b: Booking) : void
+markPendingPayment(b: Booking) : void
+markPaid(b: Booking) : void
+cancel(b: Booking) : void
+complete(b: Booking) : void
+getStateName() : String
CompletedState
+confirm(b: Booking) : void
+reject(b: Booking) : void
+markPendingPayment(b: Booking) : void
+markPaid(b: Booking) : void
+cancel(b: Booking) : void
+complete(b: Booking) : void
+getStateName() : String
Client
#id: Int
#name: String
#email: String
#role: String
-paymentMethods: List<PaymentMethod>
+login() : boolean
+logout() : void
+browseServices() : List<Service>
+requestBook(bkg1, slot, s) : Booking
+cancelBooking(b) : void
+viewBookingHistory() : List<Booking>
+viewPaymentHistory() : List<Payment>
Consultant
#id: Int
#name: String
#email: String
#role: String
-specialization: String
-isApproved: boolean
-availableSlots: List<TimeSlot>
+login() : boolean
+logout() : void
+manageAvailability(slots) : void
+acceptBooking(b) : void
+rejectBooking(b) : void
+completeBooking(b) : void
«Interface»
BookingObserver
+update(b: Booking, event: String) : void
«abstract»
User
#id: Int
#name: String
#email: String
#role: String
+login() : boolean
+logout() : void
Admin
#id: Int
#name: String
#email: String
#role: String
+login() : boolean
+logout() : void
+approveConsultant(c) : void
+rejectConsultant(c) : void
+definePolicy(p) : void
configures
PaymentMethod
-id: Int
-type: String
-details: Map<String,String>
-isDefault: boolean
+getType() : String
Service
-Id: Int
-name: String
-durationMinutes: Int
-basePrice: double
+getDetails() : String
TimeSlot
-Id: Int
-startTime: DateTime
-endTime: DateTime
-isAvailable: boolean
+markUnavailable() : void
+markAvailable() : void
ClientNotificationObserver
-client: Client
+update(b: Booking, event: String) : void
ConsultantNotificationObserver
-consultant: Consultant
+update(b: Booking, event: String) : void
AdminNotificationObserver
-admin: Admin
+update(b: Booking, event: String) : void
NotificationService
-channel: String
+sendEmail(to, msg) : void
+sendInApp(user34, msg) : void
BookingService
-db: DatabaseManager
-policy: CancellationPolicy
+createBook(bkg1, con, s, slot) : Booking
+cancelBooking(b6, client1d) : void
+getBookingHistory(client1d) : List
ConsultantService
-db: DatabaseManager
+approveConsultant(id) : void
+updateAvailability(id, slots) : void
+getAvailableSlots(id) : List
PaymentService
-db: DatabaseManager
-processor: PaymentProcessor
+processPayment(b, m) : Payment
+getPaymentHistory(client1d) : List
+issueRefund(payment1d) : void
SystemPolicy
-paymentStrategyType: String
-cancellationPolicyType: String
+getCancellationPolicy() : CancellationPolicy
+getNotificationSettings() : Map
uses
enforces
uses
uses
delegates to
defines
«Singleton»
DatabaseManager
-Instance: DatabaseManager
-connection: Connection
-DatabaseManager()
+getInstance() : DatabaseManager
+save(entity) : void
+findBy(id,type, id) : Object
+findAll(type) : List
+delete(type, id) : void
+update(entity) : void
«Interface»
CancellationPolicy
+canCancel(b: Booking) : boolean
+calculateRefund(p: Payment) : double
+getPolicyName() : String
PaymentProcessor
-strategy: PaymentStrategy
+setStrategy(p) : void
+executePayment(b, method) : Payment
uses
creates
StrictCancellationPolicy
+canCancel(b) : boolean
+calculateRefund(p) : double
+getPolicyName() : String
FlexibleCancellationPolicy
+canCancel(b) : boolean
+calculateRefund(p) : double
+getPolicyName() : String
Payment
-id: Int
-amount: double
-transactionId: String
-status: String
-timestamp: DateTime
+getTransactionId() : String
+getStatus() : String
NoCancellationPolicy
+canCancel(b) : boolean
+calculateRefund(p) : double
+getPolicyName() : String
«Interface»
PaymentStrategy
+validate(method) : boolean
+process(amount, method) : Payment
+getMethodName() : String
CreditCardStrategy
+validate(method) : boolean
+process(amount, method) : Payment
+getMethodName() : String
DebitCardStrategy
+validate(method) : boolean
+process(amount, method) : Payment
+getMethodName() : String
PayPalStrategy
+validate(method) : boolean
+process(amount, method) : Payment
+getMethodName() : String
BankTransferStrategy
+validate(method) : boolean
+process(amount, method) : Payment
+getMethodName() : String